UVM testbench conventions

This repo is a small UVM testbench for the CHIPKIT UART (loopback: driver sends on `rxd`, monitor reads `txd`). When you build an agent/env/test for another module, copy the structure and style below. The UART files are the reference; read them before writing anything new.

Layout

```
agent/         one protocol agent, everything in one package (<proto>_pkg)
environment/   env_top + scoreboard(s)
test/          test_top, test cases (tc_*), verif_pkg
top_tb/        top_tb.sv: clock, reset, interface, DUT, config_db, run_test()
run/           run.do (ModelSim/Questa compile + sim script)
```

For a second protocol, add `agent_<proto>/` (or a subfolder per agent) with its own `<proto>_pkg.svh`. Don't put two protocols in one package.

File and name conventions

Thing	Name	Example
File per class	`<proto>_<role>.svh`	`uart_driver.svh`
Sequence item	`<proto>_seqit`	`uart_seqit`
Config object	`<proto>_config`	`uart_config`
Interface	`<proto>_if`, file `<proto>_if.svh`	`uart_if`
Package	`<proto>_pkg`	`uart_pkg`
Top env / base test	`env_top`, `test_top`	
Test case	`tc_<style>_<what>`	`tc_direct_urx_utx`
Class handles	suffix `_h`	`driver_h`, `env_h`, `uart_config_h`
Module instances	suffix `_inst`	`uart_if_inst`
Virtual interface	always `vif`	
Enum types	`t_` prefix, UPPER_CASE values, explicit encodings	`t_parity`, `PARITY_ODD = 1`
Interface params	`c_` prefix, UPPER_CASE	`c_CLOCK_PERIOD_NS`
config_db keys	`<proto>_vif`, `<proto>_config`	`"uart_vif"`

- Default `name` argument in `new()` is the class name: `function new(string name = "uart_driver", uvm_component parent);`
- Factory create name matches the handle name: `driver_h = uart_driver::type_id::create("driver_h", this);`
- Always create through the factory (`::type_id::create`), never `new()`, except TLM ports.

Class skeleton

Every class follows this order. Prototypes go in the class with `extern`, bodies go below with `class::method`. Close every method with a label (`endfunction:new`, `endtask:run_phase`, `endclass:uart_monitor`).

```
class uart_monitor extends uvm_monitor;
   // UVM macro Factory registration
   `uvm_component_utils(uart_monitor)

   // Config class
   uart_config uart_config_h;
   // Virtual interface
   virtual uart_if vif;

   uvm_analysis_port #(uart_seqit) monitor_port;

   extern function new(string name = "uart_monitor", uvm_component parent);
   extern function void build_phase(uvm_phase phase);
   extern task run_phase(uvm_phase phase);
endclass:uart_monitor

function uart_monitor::new(string name = "uart_monitor", uvm_component parent);
   super.new(name, parent);
   monitor_port = new("monitor_port", this);
endfunction:new
```

- Components use `` `uvm_component_utils `` , items/sequences/config use `` `uvm_object_utils `` .
- Call `super.<phase>(phase)` first in `build_phase` and `start_of_simulation_phase` .
- Use `get_type_name()` as the report ID inside components.
- Every fatal/error/info gets an UPPER_CASE statement label, so it's greppable in the source:

```
FATAL_CONFIG_DB: `uvm_fatal("CONFIG", "cannot get() uart_config_h from uvm_config_db.")
```

Package include order

`<proto>_pkg.svh` imports `uvm_pkg` , includes `uvm_macros.svh` , then files in dependency order:

```
typedefs  ->  seqit  ->  config  ->  driver, monitor, sequencer  ->  sequence(s)  ->  agent
```

`verif_pkg.svh` imports `uvm_pkg` and every `<proto>_pkg` , then includes scoreboard(s), `env_top` , `test_top` , then every `tc_*` with relative paths (`../environment/...`). A new test case isn't visible to `+UVM_TESTNAME` until it's added here.

How config and the virtual interface flow

This is the part to get right. Nothing below the env reads the vif from config_db.

1. `top_tb` sets the vif: `uvm_config_db #(virtual uart_if)::set(null, "*", "uart_vif", uart_if_inst);` then `run_test()` .
2. `env_top::build_phase` creates the config object, gets the vif straight into `config_h.vif` , sets the config object for each consumer by instance name (below), then calls `configuration()` , a function that sets every config field, one per line, aligned.

3. Agent and scoreboard `get` the config in `build_phase`, `uvm_fatal` on failure.
4. Agent hands the config handle down in `connect_phase` by direct assignment: `driver_h.uart_config_h = uart_config_h;` (same for monitor).
5. Driver and monitor copy `vif = uart_config_h.vif;` in `start_of_simulation_phase`.

```
uvm_config_db #(uart_config)::set(this, "uart_agent_h",      "uart_config", uart_agent_config_h);
uvm_config_db #(uart_config)::set(this, "uart_scoreboard_h", "uart_config", uart_agent_config_h);
```

Pieces of an agent

typedefs.svh: every protocol option is an enum (`t_reset_polarity`, `t_data_width`, `t_parity` ...). No magic ints in config.

config (`uvm_object`): `virtual <proto>_if vif;` plus one field per option with a sane default, aligned in columns. No logic in it.

interface: only clock and reset are ports. Every DUT-facing signal is a `logic` inside. Timing constants are `parameter c_*` with a comment giving the formula. `top_tb` uses these params for clock/reset generation.

seqit: keep stimulus fields and observed fields separate (`data_arr[]` is what the driver sends, `mon_data` is what the monitor saw). Implement `do_print` and `do_compare` by hand, no field macros. `do_compare` fatals on null rhs, `$cast`s, and compares only the observed fields.

sequencer: just the factory macro and `new`. Parameterized on the seqit.

sequence: `body()` does `start_item(req)`, fill fields, `finish_item(req)`. Directed data lives here.

driver: `run_phase` is always:

```
reset_<x>_bus();   // put outputs in idle state
wait_reset();      // honour config reset_polarity
<compute timing from config>
forever begin
   seq_item_port.get_next_item(req);
   drive_data(req);
   seq_item_port.item_done();
end
```

Split protocol work into small `virtual task`s: `drive_data` (the frame), `drive_if` (one pin update + one bit time). Protocol options are handled with `case` on the config enum, with a branch per value.

monitor: `run_phase` computes timing, waits for reset, then loops `collect_data()`. `collect_data` creates a fresh seqit per transaction via the factory, samples at the middle of each bit/beat, checks every protocol field inline (start, parity, stop) with a labelled `uvm_info` on pass and a labelled `uvm_error` on fail, then `monitor_port.write(item)`. The analysis port is always named `monitor_port` and constructed in `new`.

agent: holds `driver_h`, `monitor_h`, `sequencer_h`, `<proto>_config_h`. `build_phase`: get config, create the three components. `connect_phase`: `driver_h.seq_item_port.connect(sequencer_h.seq_item_export)` and assign the config handles.

Environment

- `env_top` owns, per protocol: `<proto>_agent_h`, `<proto>_agent_config_h`, and the scoreboard. Order in `build_phase`: create config, create agent, create scoreboard, get vif, set config_db, call `configuration()`.
- `connect_phase` wires each monitor to its scoreboard: `uart_agent_h.monitor_h.monitor_port.connect(uart_scoreboard_h.observed);`
- Scoreboard uses `` `uvm_analysis_imp_decl(_observed) `` / `(_expected)` so it can take several streams, with `write_observed` / `write_expected`. Expected items sit in a queue (`expected_data[$]`), each observed item pops the front and compares with `tr.compare(exp)`. Keep `number_of_passed` and `number_of_err` counters and report in `check_phase`: error if any failed, info if any passed, error if nothing was checked at all (an empty run must not pass).

Tests

- `test_top` only creates `env_h`. All test cases extend `test_top`.
- A `tc_*` test:
- `build_phase`: `super.build_phase`, create its sequences.
- `start_of_simulation_phase`: push expected seqits into `env_h.<proto>_scoreboard_h.expected_data`.
- `run_phase`: raise objection, `seq_h.start(env_h.<proto>_agent_h.sequencer_h)`, fixed drain delay so the last frame gets through the DUT, drop objection.
- Pick the test at run time with `+UVM_TESTNAME`, never hard-code it in `run_test()`.

---

top_tb

Order: `` `include `` the interface and `verif_pkg`, import `uvm_pkg` and `verif_pkg`, clock/reset as `bit`, clock `always` from `c_CLOCK_PERIOD_NS`, reset sequence in an `initial`, interface instance, DUT instance with every port connected through `<proto>_if_inst.<signal>` (unused outputs left as `()`), then the `initial` doing config_db set and `run_test()`. Port connections are aligned in columns.

---

run.do

All paths and names are `set` variables at the top with a trailing comment. Compile order: RTL into `<dut>_lib`, then into the sim lib: `uvm_pkg.sv` (`+define+UVM_NO_DPI`), each `<proto>_pkg.svh`, then `top_tb.sv` with `+incdir+` for agent, uvm and test dirs. Sim with `-L <dut>_lib`, `+UVM_TESTNAME`, `+UVM_VERBOSITY`, `-t 1ns`. A new agent needs its own dir variable, its own `vlog` line before `top_tb`, and its dir in top_tb's `+incdir+`. Add `add wave` for the new interface instance.

If `eda_playground/build.py` is in use, add the new agent dir to `SEARCH_DIRS` and re-run it.

Style

- Indent 3 spaces in new files (the repo mixes 3 and 4; 3 is more common).
- Align `=`, `;` and port connections in columns in declaration blocks, config assignments and port maps.
- Comments are one short line above the block they describe, saying what it does or why (`// Wait until middle of start bit`). No banners, no comments restating the code.
- Times: use real time literals (`1s / real'(baud_rate)`) rather than hand-scaled numbers so it doesn't depend on the timescale.

Bugs in the reference code, don't copy them

- `uart_scoreboard`: `expected` is declared as `uvm_analysis_imp_observed` and never constructed. It should be `uvm_analysis_imp_expected` and get `new`'d in the constructor.
- `uart_monitor::run_phase` waits `@(posedge vif.resetn_i)` regardless of `reset_polarity`. Use the same `wait_reset()` logic as the driver.
- Driver computes `bit_time` as `(1/baud)*10**9` (assumes 1 ns units), monitor uses `1s/baud`. Use the `1s` form in both.
- `collect_data` has its own `forever` and is also called inside a `forever` in `run_phase`. One loop is enough.
- Monitor samples with `<=` into `mon_data`. Use blocking `=` in class code.
- Scoreboard `write_observed` creates an item and then overwrites it with `pop_front()`. Just pop. Also check the queue isn't empty before popping.
- `top_tb` sets the vif with `"*"` scope. Prefer `"uvm_test_top.env_h"` so it only reaches the env.
- `uart_driver` and `uart_scoreboard` define methods inline instead of `extern`. Use `extern` for new classes.